Assessing the Impact of Correlated Errors and Missingness on the Naive Bayes Record Linkage Algorithm

Gavin Maxwell and Dr. Dennis Tolley

2026-05-19

Introduction

In this paper, we assess the impact of the independence assumptions used to build the linkage algorithm described in David White's 1997 article, "A Review of the Statistics of Record Linkage for Genealogical Research." We conclude that, assuming data quality is reasonable, the independence assumptions need not be met in order for the algorithm to succeed.

The linkage algorithm of interest to us is derived as follows. Note that M is the event that the two records in a pair are a true match (i.e., they refer to the same person), and E_i is the event that for the i^{th} field (e.g., First Name or Birth Year) of Q fields shared by both records, a) the records agree, b) the records disagree, or c) the field is missing in one or both records.

$$
\begin{aligned}
P(M|E_1, E_2, \ldots, E_Q) &= \frac{P(E_1, E_2, \ldots, E_Q|M) \cdot P(M)}{P(E_1, E_2, \ldots, E_Q)} && \text{(Bayes Rule)} \\
&= P(M) \cdot \frac{P(E_1|M) \cdot P(E_2|M) \cdots P(E_Q|M)}{P(E_1) \cdot P(E_2) \cdots P(E_Q)} && \text{(Independence assumptions)} \\
&\propto \prod_{i=1}^{Q} \frac{P(E_i|M)}{P(E_i)} && \text{("Odds in favor of a match")} \\
\text{Log Odds} &= \sum_{i=1}^{Q} \log\left(\frac{P(E_i|M)}{P(E_i)}\right) && \text{(Sum of per-field log odds)}
\end{aligned}
$$

Note the two independence assumptions upon which the algorithm relies:

$$(1) \quad P(E_1, E_2, ..., E_Q) = \prod_{i=1}^{Q} P(E_i),$$

$$(2) \quad P(E_1, E_2, ..., E_Q|M) = \prod_{i=1}^{Q} P(E_i|M).$$

We will call (1) the mutual independence assumption and (2) the conditional independence assumption. These assumptions pertain to the field comparison outcome ("agree", "disagree", or (one or both fields) "missing") on each of the Q fields shared by a pair of records. To show the potential danger in using these assumptions for record linkage, we will introduce parameters controlling 1) the marginal probabilities

of recording errors and missingness, and 2) the correlation between the events E_i (field comparison outcomes). We show that as the parameters controlling correlation become more extreme, the independence assumptions are increasingly inaccurate. Finally, we demonstrate the durability of the linkage algorithm as those parameters change.

Step 0: Generate an "Anchor" Genealogical File

Before we can make record linkage decisions, we need record pairs. Organizations that perform record linkage may be interested in linking a raw dataset to an existing database. In this "anchor" scenario, the existing database (the anchor) is assumed to carry few errors, while the target dataset may contain many errors. Accordingly, we first generate a synthetic genealogical anchor file which we will call "File A." It will represent a sample of 1,000 males from Utah in 1880. We will choose a few common names and several rare ones, reflecting a realistic property of distributions of names. We will assume that the records in the file reflect the truth about the fake individuals to whom they refer (i.e., its fields are free of recording errors and missingess). Importantly, we keep track of the individual to whom each record refers by assigning each record a unique "Identity" integer, which we will preserve in the messy dataset to be generated later. This column ultimately allows us to evaluate the performance of the linkage algorithm by distinguishing between true matches and true non-matches in a pool of record pairs.

```
# 1. First Names Pool (blends the English monoliths, Scandinavian imports, and
# pioneer cultural names)
first_names <- c("John", "William", "James", "Joseph", "George",
                 "Thomas", "Charles", "Brigham", "Heber", "Hyrum",
                 "Jens", "Niels", "Hans", "Ephraim", "Orson",
                 "Parley", "Lorenzo", "Erastus", "Wilford", "Moroni")

# Creating a steep power-law distribution for first names
# The first few names are incredibly common; the rest trail off.
fn_probs <- c(0.12, 0.12, 0.10, 0.10, 0.08, 0.08, 0.07, 0.06, 0.05, 0.04,
              0.03, 0.03, 0.02, 0.02, 0.02, 0.015, 0.015, 0.01, 0.01, 0.01)

# 2. Last Names Pool (Heavy English, Welsh, and Scandinavian influence)
last_names <- c("Smith", "Johnson", "Christensen", "Jensen", "Jones",
                "Hansen", "Williams", "Taylor", "Brown", "Anderson",
                "Larsen", "Davis", "Richards", "Nielsen", "Evans",
                "Madsen", "Snow", "Pratt", "Kimball", "Farr")

# Creating a similar power-law distribution for last names
ln_probs <- c(0.15, 0.10, 0.09, 0.09, 0.08, 0.07, 0.06, 0.05, 0.05, 0.04,
              0.04, 0.03, 0.03, 0.03, 0.02, 0.02, 0.015, 0.015, 0.01, 0.01)

# 3. Utah Counties in 1880
# (Weighted roughly by historical population density)
counties <- c("Salt Lake", "Utah", "Weber", "Cache", "Sanpete",
              "Davis", "Box Elder", "Washington", "Juab", "Tooele")

county_probs <- c(0.25, 0.15, 0.12, 0.10, 0.10,
                  0.08, 0.06, 0.05, 0.05, 0.04)

# 4. Generate the Anchor File (N = 1000)
set.seed(1880) # For exact reproducibility
fileA <- data.frame(
  Identity = 1:1000,
```

```
  Field1 = sample(first_names, 1000, replace = TRUE, prob = fn_probs),
  Field2 = sample(last_names, 1000, replace = TRUE, prob = ln_probs),
  Field3 = sample(counties, 1000, replace = TRUE, prob = county_probs),
  stringsAsFactors = FALSE
)

# Preview the historical distribution
head(fileA, 10)
```

```
##    Identity  Field1      Field2     Field3
## 1         1 Charles Christensen       Utah
## 2         2   James      Larsen      Weber
## 3         3   James     Johnson  Salt Lake
## 4         4  Thomas      Larsen Washington
## 5         5  Thomas      Jensen       Utah
## 6         6 Brigham      Jensen      Davis
## 7         7    Hans       Brown  Salt Lake
## 8         8    John    Richards       Utah
## 9         9    John Christensen       Utah
## 10       10  Joseph      Jensen      Weber
```

Step 1: Parameterize Dependence of Field Comparison Outcomes

We are interested in breaching the assumptions of independence that pertain to field comparison outcomes within a record pair. To do so, we will perturb File A to yield File B such that when records are paired across files, the independence assumptions are not true. Using the `rmvbin()` function of the `bindata` package, we will draw from a Multivariate Bernoulli distribution to simulate A) incorrectness and missingness in recording (we will assume these processes occur independently), and B) correlation in such incorrectness and missingness between fields in a record pair.

The first helper function below, `draw_multivariate()`, repeatedly draws from a Multivariate Bernoulli distribution. We execute this function twice: first to introduce recording errors into File B, and second to independently introduce missingness. The function accepts four arguments: n, the number of records to generate; Q, the number of fields being evaluated (representing the intersection of fields available in both files); lambda, a probability or vector of probabilities defining the marginal rate of the target event (an error or a missing value) for each field; and rho, a scalar or matrix representing the correlation of those events between fields. It is important to note that if varying marginal probabilities (λ) are assigned across fields, the theoretical maximum correlation (ρ) between those fields is strictly bound below 1.0. Consequently, not every arbitrary combination of λ and ρ is mathematically feasible. This constraint is naturally avoided if uniform marginal probabilities are applied across fields, which allows the correlation between them to reach the absolute maximum of 1.0.

The second helper function below, `substitute_strings()`, is executed wherever the result of draw_multivariate dictates that a recording error take place. It replaces the value from File A with a different value from the pool of possible values for the field. For simplicity, we assume that for each field, this pool consists only of the unique, non-missing values observed for that field. While swaps are not the only type of possible recording error (typographical errors or OCR failures are also common), using swaps for every recording error suits our purposes because field comparison outcomes are limited to "agree", "disagree", and "missing" in the linkage algorithm.

The main perturbation pipeline, `perturb_dataset()`, generates the messy target data (File B) by simulating real-world degradation against the anchor file. It applies the helper functions sequentially, ensuring that physical missingness appropriately supersedes any generated recording errors. The function accepts the anchor dataframe alongside four parameters to control the degradation: λ_v and λ_m, scalars or vectors

representing the marginal probability of a recording error and the marginal probability of missingness, respectively; and ρ_v and ρ_m, scalars or matrices representing correlation of recording errors across and the correlation of missingness across fields, respectively.

```r
# 1. Helper Function: Standardized Multivariate Draw
draw_multivariate <- function(n, Q, lambda, rho) {
  if (length(lambda) == 1) lambda_vec <- rep(lambda, Q)
  else lambda_vec <- lambda

  if (length(rho) == 1) {
    corr_matrix <- matrix(rho, nrow = Q, ncol = Q)
    diag(corr_matrix) <- 1
  } else {
    corr_matrix <- rho
  }

  return(rmvbin(n = n, margprob = lambda_vec, bincorr = corr_matrix))
}

# 2. Helper Function: Force String Substitution
# Ensures the new drawn value is genuinely different from the original value
substitute_strings <- function(original_values, pool) {
  new_values <- sample(pool, length(original_values), replace = TRUE)

  # Identify any accidental exact matches and redraw them
  same_idx <- which(new_values == original_values)
  while (length(same_idx) > 0) {
    new_values[same_idx] <- sample(pool, length(same_idx), replace = TRUE)
    same_idx <- which(new_values == original_values)
  }

  return(new_values)
}

# 3. Main Perturbation Pipeline
perturb_dataset <- function(df, lambda_v, rho_v,
                            lambda_m, rho_m)
                                          {
  n <- nrow(df)
  fields_to_perturb <- setdiff(names(df),"Identity")
  Q <- length(fields_to_perturb)
  df_perturbed <- df

  # ========================================
  # PROCESS 1: Generate Incorrect Values (Typos/Swaps)
  # ========================================
  val_matrix <- draw_multivariate(n, Q, lambda_v, rho_v)

  for (j in 1:Q) {
    field <- fields_to_perturb[j]
    err_idx <- which(val_matrix[, j] == 1)

    if (length(err_idx) > 0) {
      # Define the pool of all possible values for this specific field
```

```
      field_pool <- unique(df[[field]][!is.na(df[[field]])])

      # Apply the substitution
      df_perturbed[[field]][err_idx] <- substitute_strings(
        original_values = df[[field]][err_idx],
        pool = field_pool
      )
    }
  }

  # ========================================
  # PROCESS 2: Generate Missingness
  # ========================================
  mis_matrix <- draw_multivariate(n, Q, lambda_m, rho_m)

  for (j in 1:Q) {
    field <- fields_to_perturb[j]
    mis_idx <- which(mis_matrix[, j] == 1)

    # Overwrite with NA
    if (length(mis_idx) > 0) {
      df_perturbed[[field]][mis_idx] <- NA_character_
    }
  }

  return(df_perturbed)
}
```

Step 2: Generate Files "B" Under Varying Parameters and Pair Records

We desire to generate various sets of record pairs that vary according to how well they suit White's independence assumptions. To do so, we will apply the parameters created in step 1 to generate various messy versions (Files "B") of the anchor file, File A. Since we assume File A to be entirely correct, any recording errors and missing fields in File B will map directly to disagreement and missingness, respectively, in the corresponding field comparison of the true match record pair. Each of the new files, File "B1", File "B2", etc., will reflect a different degree of correlation in the events of recording errors and missingness between fields within a record. File B1 will have zero correlation (such that ρ_v and ρ_m equal zero) and File B5 will have the most correlation (such that ρ_v and ρ_m equal 0.8). We ignore negative values of ρ_v and ρ_m, focusing only on the possibly of zero or positive correlation in the event of recording errors or missingness between fields in a record. And to ensure that ρ_v and ρ_m are allowed to be large, we will assign uniform probabilities of recording errors (λ_v) and missingness (λ_m) across fields. We arbitrarily choose λ_v to equal 0.2 and λ_m to equal 0.1. These values are neither very low nor very high and indicate reasonable data quality. A similar process can be followed if a different choice of λ_v and λ_m is of interest.

After generating five versions of File "B," we cross join File A with each version of File B, yielding five sets of 1,000 x 1,000 = 1,000,000 record pairs. For each pair, we use the "Identity" columns to track the true match status (yes or no), and we further evaluate and store the field comparison outcomes ("agree", "disagree", "missing") on each of the three shared fields.

```
# 1. Generate Files B Under Various Sets of Parameters

fileB1 <- perturb_dataset(fileA, lambda_v = .2, lambda_m = .1,
                                 rho_v = 0, rho_m = 0)
```

```r
fileB2 <- perturb_dataset(fileA, lambda_v = .2, lambda_m = .1,
                                 rho_v = .2, rho_m = .2)

fileB3 <- perturb_dataset(fileA, lambda_v = .2, lambda_m = .1,
                                 rho_v = .4, rho_m = .4)

fileB4 <- perturb_dataset(fileA, lambda_v = .2, lambda_m = .1,
                                 rho_v = .6, rho_m = .6)

fileB5 <- perturb_dataset(fileA, lambda_v = .2, lambda_m = .1,
                                 rho_v = .8, rho_m = .8)

# 2. Write Function to Pair Records, Track Match Status, and Compare Fields
# (i.e., evaluate E_i)

pair_records <- function(fileA, fileB){
# Create the Cartesian product using cross_join (requires dplyr 1.1.0+)
record_pairs <- cross_join(fileA, fileB, suffix = c("_A", "_B")) %>%
  mutate(
    # Determine true match status
    True_Match = ifelse(Identity_A == Identity_B, "Yes", "No"),

    # Evaluate Field 1 Agreement Status for Each Pair
    Field1_Status = case_when(
      is.na(Field1_A) | is.na(Field1_B) ~ "Missing",
      Field1_A == Field1_B ~ "Agree",
      TRUE ~ "Disagree"
    ),

    # Evaluate Field 2 Agreement Status for Each Pair
    Field2_Status = case_when(
      is.na(Field2_A) | is.na(Field2_B) ~ "Missing",
      Field2_A == Field2_B ~ "Agree",
      TRUE ~ "Disagree"
    ),

    # Evaluate Field 3 Agreement Status for Each Pair
    Field3_Status = case_when(
      is.na(Field3_A) | is.na(Field3_B) ~ "Missing",
      Field3_A == Field3_B ~ "Agree",
      TRUE ~ "Disagree"
    ),
  )

  return(record_pairs)
}

# 3. Pair Records, Track Match Status, and Compare Fields

record_pairs_AB1 <- pair_records(fileA, fileB1)

record_pairs_AB2 <- pair_records(fileA, fileB2)
```

```
record_pairs_AB3 <- pair_records(fileA, fileB3)

record_pairs_AB4 <- pair_records(fileA, fileB4)

record_pairs_AB5 <- pair_records(fileA, fileB5)
```

Step 3: Assess Violation of the Independence Assumptions

To check that the independence assumptions are violated as ρ_v and ρ_m increase, we build contingency tables that compare the outcomes expected under the independence assumptions to the outcomes actually observed. For each contingency table, we return the Chi-squared statistic, which serves as a metric of the severity of the deviation of the actual counts from the expected counts. The following code contains helper functions to create a contingency table pertaining to each independence assumption. It also contains a helper function to return the Chi-squared statistic and corresponding p-value for a given contingency table. The code executes the helper functions to return the contingency tables arising from the independent file (FileB1) and the highly correlated file (FileB5).

```
# 1. Write helper function that computes Pearson chi-square statistic
compute_table_metric <- function(actual, expected) {
  # Calculate Degrees of Freedom dynamically based on the input table dimensions
  # Formula: prod(dims) - 1 - sum(dims - 1)
  dims <- dim(actual)
  df <- prod(dims) - 1 - sum(dims - 1)

  # Flatten the 3D arrays into 1D vectors for straightforward vector math
  act <- as.vector(actual)
  exp <- as.vector(expected)

  # Chi-Square Statistic
  # We add a tiny epsilon (1e-9) to the denominator to prevent division-by-zero
  # errors in perfectly clean, zero-expectation cells.
  chi_sq_val <- sum(((act - exp)^2) / (exp + 1e-9))

  # Calculate the p-value (probability of seeing this Chi-Sq or larger assuming
  # independence)
  p_val <- pchisq(chi_sq_val, df = df, lower.tail = FALSE)

  return(list(ChiSquare = chi_sq_val, p_value = p_val, DF = df))
}

# 2. Write Function to Generate 3D Contingency Tables (Actual vs Expected) to
# Test Mutual Independence

# This time, we will not subset to true matches; we desire to test
# overall/unconditional independence)

generate_mutual_3D_contingency <- function(record_pairs, quiet = FALSE) {

  N <- nrow(record_pairs)

  # Ensure all levels are present so zero-count cells aren't dropped
  lvls <- c("Agree", "Disagree", "Missing")
```

```
  f1 <- factor(record_pairs$Field1_Status, levels = lvls)
  f2 <- factor(record_pairs$Field2_Status, levels = lvls)
  f3 <- factor(record_pairs$Field3_Status, levels = lvls)

  # 1. Calculate Actual 3D Table for ALL pairs
  actual <- table(Field1 = f1, Field2 = f2, Field3 = f3)

  # 2. Calculate Overall Marginal Probabilities
  p1 <- prop.table(table(f1))
  p2 <- prop.table(table(f2))
  p3 <- prop.table(table(f3))

  # 3. Calculate Expected 3D Table assuming complete mutual independence
  expected <- N * (p1 %o% p2 %o% p3)

  # 4. Format the array to hold "Actual (Expected)" strings
  formatted <- array(
    sprintf("%d (%.1f)", actual, expected),
    dim = dim(actual),
    dimnames = dimnames(actual)
  )

  # 5. Compute chi-squared statistic
  metrics <- compute_table_metric(actual, expected)

  # 6. Print the results cleanly if quiet = FALSE (its default value)
  if (quiet == FALSE) {
  cat("\n=========================================================\n")
  cat(sprintf("Overall Joint Distribution (All Pairs, N = '%s')\n", N))
  cat(sprintf("Metrics -> Chi-Sq: %.2f | p-val: %.2f | df: %.2f\n",
              metrics$ChiSquare, metrics$p_value, metrics$DF))
  cat("Format: Actual (Expected)\n")
  cat("=========================================================\n\n")

  print(ftable(formatted))
  }

    # 7. Return only the Chi-Square statistic if quiet = TRUE
  if (quiet == TRUE) {
  metrics$ChiSquare
  }
}

# 3. Write Function to Generate 3D Contingency Tables (Actual vs Expected) to
# Test Conditional Independence

generate_conditional_3D_contingency <- function(record_pairs,
                                                match_status = "Yes",
                                                quiet = FALSE) {

  # Subset to the requested match status
  sub_df <- subset(record_pairs, True_Match == match_status)
  N <- nrow(sub_df)
```

```
  # Ensure all levels are present so zero-count cells aren't dropped
  lvls <- c("Agree", "Disagree", "Missing")
  f1 <- factor(sub_df$Field1_Status, levels = lvls)
  f2 <- factor(sub_df$Field2_Status, levels = lvls)
  f3 <- factor(sub_df$Field3_Status, levels = lvls)

  # 1. Calculate Actual 3D Table
  actual <- table(Field1 = f1, Field2 = f2, Field3 = f3)

  # 2. Calculate Marginal Probabilities
  p1 <- prop.table(table(f1))
  p2 <- prop.table(table(f2))
  p3 <- prop.table(table(f3))

  # 3. Calculate Expected 3D Table assuming complete independence
  # The %o% operator calculates the outer product of the arrays
  expected <- N * (p1 %o% p2 %o% p3)

  # 4. Format the array to hold "Actual (Expected)" strings
  formatted <- array(
    sprintf("%d (%.1f)", actual, expected),
    dim = dim(actual),
    dimnames = dimnames(actual)
  )

  # 5. Compute chi-squared statistic
  metrics <- compute_table_metric(actual, expected)

  # 6. Print the results cleanly if quiet = FALSE (its default value)
  if (quiet == FALSE) {
  cat("\n=========================================================\n")
  cat(sprintf("Joint Distribution for True_Match = '%s' (N = %d)\n",
              match_status, N))
  cat(sprintf("Metrics -> Chi-Sq: %.2f | p-val: %.2f | df: %.2f\n",
              metrics$ChiSquare, metrics$p_value, metrics$DF))
  cat("Format: Actual (Expected)\n")
  cat("=========================================================\n\n")

  print(ftable(formatted))
  }

  # 7. Return only the Chi-Square statistic if quiet = TRUE
    if (quiet == TRUE) {
  metrics$ChiSquare
  }
}

# 4. Run the mutual independence evaluation on the independent file
# (rho_v=0, rho_m=0)
generate_mutual_3D_contingency(record_pairs_AB1)
```

```
##
## =====================================================
```

```
## Overall Joint Distribution (All Pairs, N = '1000000')
## Metrics -> Chi-Sq: 7140.93 | p-val: 0.00 | df: 20.00
## Format: Actual (Expected)
## ======================================================
##
##                   Field3             Agree          Disagree           Missing
## Field1   Field2
## Agree    Agree           881 (488.9)       3446 (3307.7)     394 (389.3)
##          Disagree        6415 (6322.8)     43335 (42779.9)   4657 (5034.8)
##          Missing         761 (773.7)       4573 (5234.9)     486 (616.1)
## Disagree Agree           6242 (6368.3)     42696 (43087.9)   5095 (5071.0)
##          Disagree        82899 (82364.2)   556086 (557277.1) 65854 (65586.2)
##          Missing         10272 (10078.7)   69394 (68192.9)   7514 (8025.6)
## Missing  Agree           666 (669.9)       4538 (4532.6)     491 (533.4)
##          Disagree        8213 (8664.3)     60583 (58622.5)   5509 (6899.3)
##          Missing         442 (1060.2)      5558 (7173.5)     3000 (844.3)
```

```
# 5. Run the mutual independence evaluation on the highly dependent file
# (rho_v=.8,rho_m=.8)
generate_mutual_3D_contingency(record_pairs_AB5)
```

```
##
## ======================================================
## Overall Joint Distribution (All Pairs, N = '1000000')
## Metrics -> Chi-Sq: 5470352.56 | p-val: 0.00 | df: 20.00
## Format: Actual (Expected)
## ======================================================
##
##                   Field3             Agree          Disagree           Missing
## Field1   Field2
## Agree    Agree           1139 (445.8)      3761 (3053.4)     25 (428.1)
##          Disagree        7252 (5925.0)     48874 (40579.5)   379 (5689.1)
##          Missing         98 (755.4)        845 (5173.5)      402 (725.3)
## Disagree Agree           7270 (5832.2)     47843 (39944.4)   519 (5600.1)
##          Disagree        95030 (77510.6)   650831 (530862.9) 6077 (74425.0)
##          Missing         925 (9881.8)      9132 (67679.6)    3598 (9488.4)
## Missing  Agree           87 (823.8)        733 (5642.2)      1184 (791.0)
##          Disagree        1107 (10948.6)    10073 (74985.7)   11816 (10512.7)
##          Missing         611 (1395.8)      5389 (9559.9)     85000 (1340.3)
```

```
# 6. Run the conditional independence evaluation on the independent file
# (rho_v=0,rho_m=0)
generate_conditional_3D_contingency(record_pairs_AB1, match_status = "Yes")
```

```
##
## ======================================================
## Joint Distribution for True_Match = 'Yes' (N = 1000)
## Metrics -> Chi-Sq: 17.45 | p-val: 0.62 | df: 20.00
## Format: Actual (Expected)
## ======================================================
##
##                   Field3       Agree    Disagree     Missing
## Field1   Field2
```

```
## Agree    Agree           405 (394.8) 80 (86.5)   52 (49.4)
##          Disagree        82 (90.2)   23 (19.8)   10 (11.3)
##          Missing         54 (55.1)   13 (12.1)   7 (6.9)
## Disagree Agree           96 (100.6)  19 (22.0)   10 (12.6)
##          Disagree        30 (23.0)   7 (5.0)     4 (2.9)
##          Missing         15 (14.0)   3 (3.1)     1 (1.8)
## Missing  Agree           49 (48.4)   15 (10.6)   5 (6.1)
##          Disagree        9 (11.1)    1 (2.4)     1 (1.4)
##          Missing         4 (6.8)     2 (1.5)     3 (0.8)
```

```
# 7. Run the conditional independence evaluation on the highly dependent file
# (rho_v=.8,rho_m=.8)
generate_conditional_3D_contingency(record_pairs_AB5, match_status = "Yes")
```

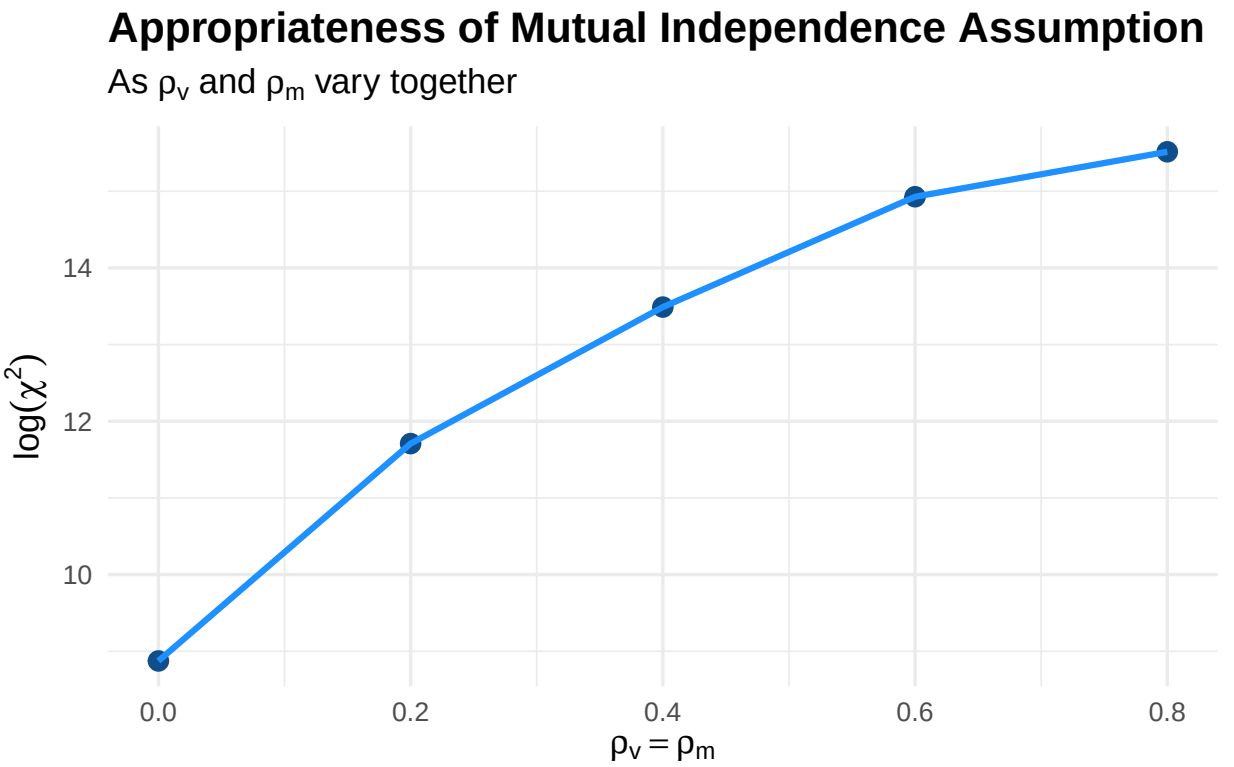
```
##
## ======================================================
## Joint Distribution for True_Match = 'Yes' (N = 1000)
## Metrics -> Chi-Sq: 8545.63 | p-val: 0.00 | df: 20.00
## Format: Actual (Expected)
## ======================================================
##
##                   Field3       Agree    Disagree     Missing
## Field1   Field2
## Agree    Agree           628 (344.0) 15 (92.0)   5 (53.3)
##          Disagree        20 (96.6)   18 (25.8)   0 (15.0)
##          Missing         11 (52.2)   0 (14.0)    4 (8.1)
## Disagree Agree           17 (89.8)   10 (24.0)   2 (13.9)
##          Disagree        13 (25.2)   141 (6.7)   0 (3.9)
##          Missing         0 (13.6)    0 (3.6)     0 (2.1)
## Missing  Agree           10 (56.9)   1 (15.2)    10 (8.8)
##          Disagree        1 (16.0)    0 (4.3)     3 (2.5)
##          Missing         3 (8.6)     3 (2.3)     85 (1.3)
```

From the contingency tables for both independence assumptions, the actual counts are much closer to the expected counts for the record pairs arising from File B1 (the independent baseline) than File B5 (for which correlation in recording errors and missingness was most extreme). Note that the large chi-squared statistic and associated low p-value for File B1 under the mutual independence assumption can be attributed to two factors. First, since our sample size is so large (N = 1,000,000), the chi-squared test has extremely high statistical power: even minor deviations from the expected counts lead to low p-values. Second, the cluster of 1,000 true matches in the population of 1,000,000 record pairs. Though they are heavily outnumbered, the chance of repeated agreement across fields for true matches is much higher than for true non-matches. This drives the agree/agree/agree cell count much higher than expected under mutual independence (which is computed using marginal probabilities from the population).

The pattern in chi-squared statistics across all versions of File B confirms the trend hinted at by the contingency tables. Notice from the log-scaled graphs that for both independence assumptions, the chi-squared statistic increases almost exponentially as the correlation in recording errors and missingness grows (the Files "B" are labeled such that the nth version of File B, File "Bn", was generated using $\rho_v = \rho_m = 0.2n - 0.2$). This means the assumptions of mutual independence and conditional independence do not suit a genealogical file for which ρ_v and ρ_m are high.

```
# Store the Mutual Independence Results for Plotting
mut_chi1 <- generate_mutual_3D_contingency(record_pairs_AB1, quiet = TRUE)
mut_chi2 <- generate_mutual_3D_contingency(record_pairs_AB2, quiet = TRUE)
```

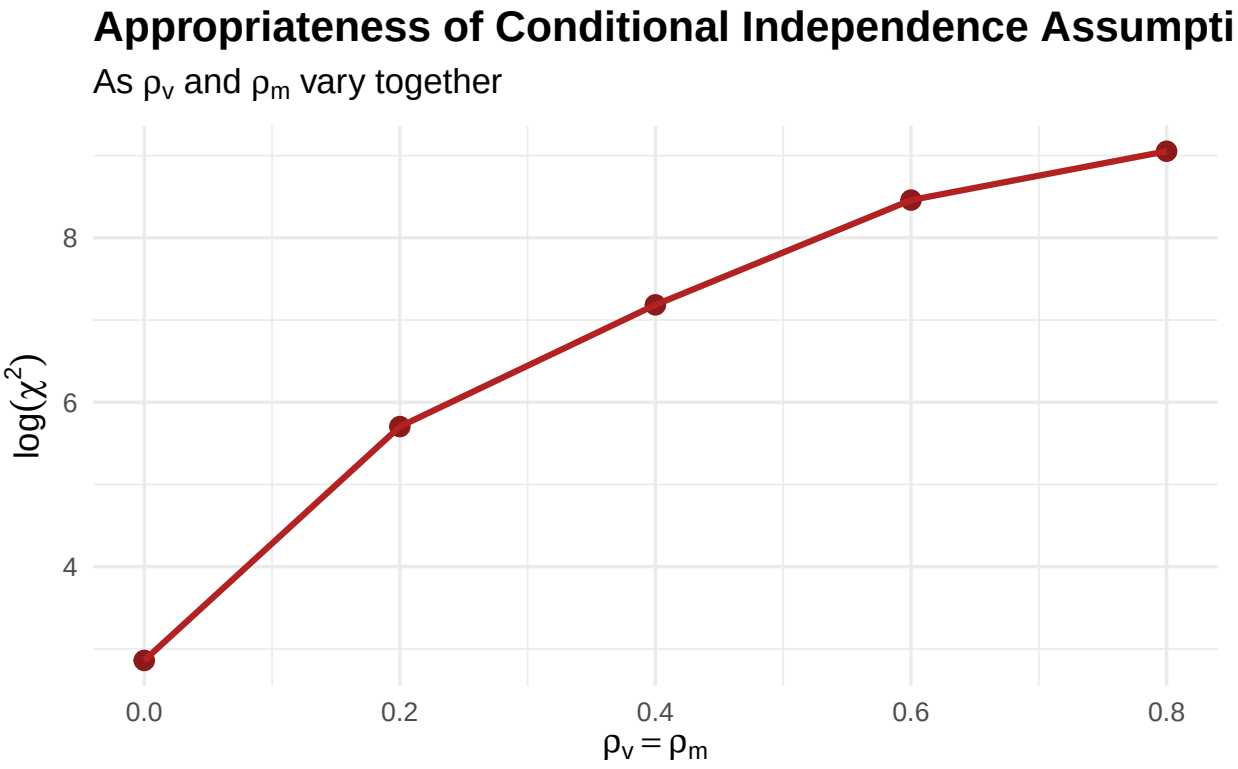
```r
mut_chi3 <- generate_mutual_3D_contingency(record_pairs_AB3, quiet = TRUE)
mut_chi4 <- generate_mutual_3D_contingency(record_pairs_AB4, quiet = TRUE)
mut_chi5 <- generate_mutual_3D_contingency(record_pairs_AB5, quiet = TRUE)

mutual_ind <- data.frame(File_B = c("File B1", "File B2", "File B3",
                                    "File B4", "File B5"),
                         Rho = c(0, .2, .4, .6, .8),
                         Chi_sq = c(mut_chi1, mut_chi2, mut_chi3,
                                    mut_chi4, mut_chi5))

# Mutual Independence Plot
ggplot(mutual_ind, aes(x = Rho, y = log(Chi_sq))) +
  geom_point(size = 3, color = "dodgerblue4") +
  geom_line(linewidth = 1.1, color = "dodgerblue") +
  labs(
    x = expression(rho[v] == rho[m]),
    y = expression(log(chi^2)),
    title = "Appropriateness of Mutual Independence Assumption",
    subtitle = expression("As " * rho[v] * " and " * rho[m] * " vary together"),
    caption = stringr::str_wrap("Large chi-squared statistic indicates greater
                                deviation between observed and expected counts
                                under independence.", width = 110)
  ) +
  theme_minimal(base_size = 13) +
  theme(
    plot.title = element_text(face = "bold"),
    axis.title = element_text(face = "bold"),
    plot.caption = element_text(hjust = 0)
  )
```



Large chi−squared statistic indicates greater deviation between observed and expected counts u
independence.

```
# Store the Conditional Independence Results for Plotting
cond_chi1 <- generate_conditional_3D_contingency(record_pairs_AB1, quiet = TRUE)
cond_chi2 <- generate_conditional_3D_contingency(record_pairs_AB2, quiet = TRUE)
cond_chi3 <- generate_conditional_3D_contingency(record_pairs_AB3, quiet = TRUE)
cond_chi4 <- generate_conditional_3D_contingency(record_pairs_AB4, quiet = TRUE)
cond_chi5 <- generate_conditional_3D_contingency(record_pairs_AB5, quiet = TRUE)

conditional_ind <- data.frame(File_B = c("File B1", "File B2", "File B3",
                                         "File B4", "File B5"),
                              Rho = c(0, .2, .4, .6, .8),
                              Chi_sq = c(cond_chi1, cond_chi2, cond_chi3,
                                         cond_chi4, cond_chi5))

# Conditional Independence Plot
ggplot(conditional_ind, aes(x = Rho, y = log(Chi_sq))) +
  geom_point(size = 3, color = "firebrick4") +
  geom_line(linewidth = 1.1, color = "firebrick") +
  labs(
    x = expression(rho[v] == rho[m]),
    y = expression(log(chi^2)),
    title = "Appropriateness of Conditional Independence Assumption",
    subtitle = expression("As " * rho[v] * " and " * rho[m] * " vary together"),
    caption = stringr::str_wrap("Large chi-squared statistic indicates greater
                                 deviation between observed and expected counts
                                 under independence.", width = 110)
  ) +
```

```
theme_minimal(base_size = 13) +
theme(
  plot.title = element_text(face = "bold"),
  axis.title = element_text(face = "bold"),
  plot.caption = element_text(hjust = 0)
)
```



Step 4: Evaluate the Linkage Algorithm As Parameters Change

Our results from Step 3 tell us that the parameters we created in Step 1 to generate messy files in Step 2 work as intended. That is, the parameters ρ_v and ρ_m used (in conjunction with λ_v and λ_m) to generate File B from a true File A can be used to violate the independence assumptions underlying the algorithm used to link the records of File A with those of File B. In Step 4, we attempt to answer the question, "How well does the algorithm work across different values of ρ_v and ρ_m?"

To evaluate the performance of the linkage algorithm, we will use the criterion of recall at 95% precision, which we compute via 100 simulations across a grid of correlation pairs (ρ_v, ρ_m) on [0,.80] x [0,.80]. Recall is the proportion of all true matches that we tag as matches using the algorithm. It answers the question, "Of all the true matches in the set of record pairs, what proportion did we successfully find?" Precision is the proportion of all tagged record pairs that are true matches. It answers the question, "Of all the record pairs that we tagged, what proportion are true matches?" Of course, we would like to have both high recall and high precision, but they move in opposite directions. Thus, we fix precision at 95% to reduce false positives and use the recall at that boundary to assess how well the algorithm performs across the region of parameter choices. We prevent correlation parameters from exceeding 0.80 to ensure they are mathematically compatible with the previously selected marginal probabilities $\lambda_v = 0.2$ and $\lambda_m = 0.1$. Further, we manually

change the ratio of true non-matches to true matches from 999:1 to 19:1, which reduces the crippling effect on the algorithm of accidental field coincidences in true non-matches.

```
n_simulations <- 100
cat(sprintf("Starting %d-Iteration Monte Carlo Simulation...\n", n_simulations))
```

```
## Starting 100-Iteration Monte Carlo Simulation...
```

```
cat("This will take a couple of hours.\n")
```

```
## This will take a couple of hours.
```

```
# 1. Define constants
resolution <- 21
lambda_v <- 0.2
lambda_m <- 0.1

# 2. Downsample and pre-calculate the pairing identities (speed up computation)

# ============================================================================
# DOWNSAMPLE: SIMULATE THE OUTCOME OF BLOCKING WITHOUT ACTUALLY BLOCKING
#
# In a production environment, computational limits are managed using blocking
# keys to filter out massive swaths of True Non-Matches.
# To mimic this post-filtering environment, we strategically downsample the
# true Non-Matches to achieve a 19:1 (U to M) prevalence ratio.
# ============================================================================

# We assume all 1,000 True Matches survive blocking
M_identities <- data.frame(Identity_A = 1:1000, Identity_B = 1:1000,
                           True_Match = "Yes")

set.seed(42) # For reproducible downsampling
U_identities <- data.frame(
  Identity_A = sample(1:1000, 25000, replace = TRUE),
  Identity_B = sample(1:1000, 25000, replace = TRUE)
) %>%
  filter(Identity_A != Identity_B) %>%
  slice_head(n = 19000) %>%
  mutate(True_Match = "No")

# The unified 20,000 row pairing skeleton
blocked_pairs_skeleton <- bind_rows(M_identities, U_identities)

# 3. Loop: Generate, Evaluate, Store, and Aggregate Results
# List to hold the results of each full grid pass
all_simulation_results <- list()

# --- THE MONTE CARLO OUTER LOOP ---
for(sim in 1:n_simulations) {
#  cat(sprintf("\n--- Running Simulation Pass %d of %d ---\n", sim, n_simulations))
```

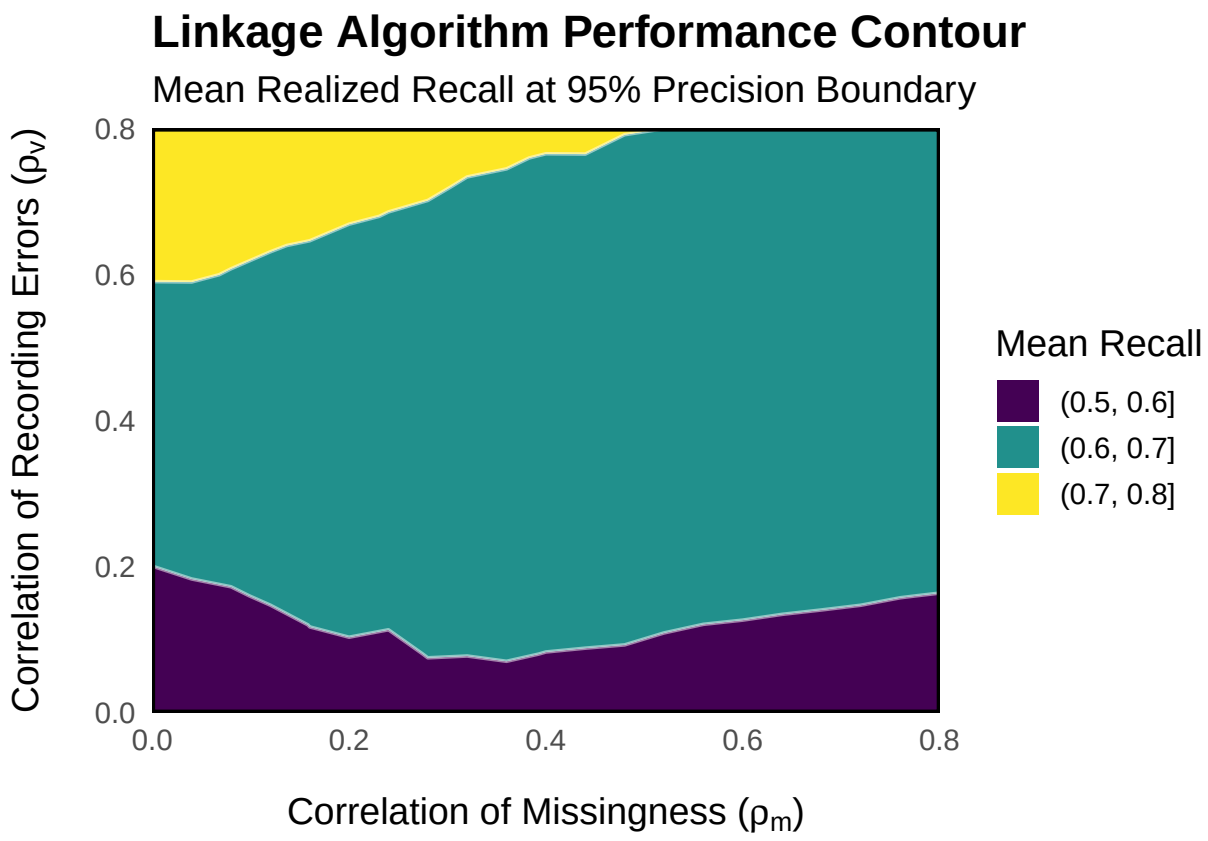
```r
# Results grid fresh for this iteration
rho_v_seq <- seq(0,.80,length.out=resolution)
rho_m_seq <- seq(0,.80,length.out=resolution)
current_grid <- expand.grid(rho_v = rho_v_seq, rho_m = rho_m_seq)
current_grid$Recall <- NA
current_grid$Iteration <- sim

for(i in 1:nrow(current_grid)) {
  r_v <- current_grid$rho_v[i]
  r_m <- current_grid$rho_m[i]

  # A. Generate the messy File B
  fileB_temp <- suppressWarnings(perturb_dataset(fileA,lambda_v,r_v,lambda_m,r_m))
  # Suppress harmless common warning: regularize.values(x, y, ties,
  # missing(ties), na.rm = na.rm) : collapsing to unique 'x' values

  # B. Join the fields to our 20,000 paired identities
  eval_df <- blocked_pairs_skeleton %>%
    left_join(fileA, by = c("Identity_A" = "Identity")) %>%
    left_join(fileB_temp, by = c("Identity_B" = "Identity"),
              suffix = c("_A", "_B")) %>%
    # Evaluate comparisons
    mutate(
      F1 = case_when(is.na(Field1_A) | is.na(Field1_B) ~ "Missing",
                     Field1_A == Field1_B ~ "Agree", TRUE ~ "Disagree"),
      F2 = case_when(is.na(Field2_A) | is.na(Field2_B) ~ "Missing",
                     Field2_A == Field2_B ~ "Agree", TRUE ~ "Disagree"),
      F3 = case_when(is.na(Field3_A) | is.na(Field3_B) ~ "Missing",
                     Field3_A == Field3_B ~ "Agree", TRUE ~ "Disagree")
    )

  # C. EMPIRICAL PARAMETER ESTIMATION
  # Helper function to get exact dataset probabilities for P(E_i|M) and P(E_i)
  get_probs <- function(field_vec, match_vec) {
    # Isolate True Matches
    M_vec <- field_vec[match_vec == "Yes"]

    # Calculate empirical probabilities
    p_M <- table(factor(M_vec, levels = c("Agree", "Disagree", "Missing"))) /
      length(M_vec)
    p_All <- table(factor(field_vec, levels=c("Agree","Disagree","Missing"))) /
      length(field_vec)

    return(list(M = as.numeric(p_M), All = as.numeric(p_All)))
  }

  probs_F1 <- get_probs(eval_df$F1, eval_df$True_Match)
  probs_F2 <- get_probs(eval_df$F2, eval_df$True_Match)
  probs_F3 <- get_probs(eval_df$F3, eval_df$True_Match)

  # Compute the vector of possible log odds scores for each field
  # Rows: 1=Agree, 2=Disagree, 3=Missing
  # Log base 2 is standard in information theory
```

```r
  weight_map_F1 <- log2(probs_F1$M / probs_F1$All)
  weight_map_F2 <- log2(probs_F2$M / probs_F2$All)
  weight_map_F3 <- log2(probs_F3$M / probs_F3$All)

  # D. Compute Log Odds scores
  # Convert status strings to vector indices (Agree=1, Disagree=2, Missing=3)
  idx_F1 <- match(eval_df$F1, c("Agree", "Disagree", "Missing"))
  idx_F2 <- match(eval_df$F2, c("Agree", "Disagree", "Missing"))
  idx_F3 <- match(eval_df$F3, c("Agree", "Disagree", "Missing"))

  # Sum the independent log-odds for the 3 fields (using White's independence
  # assumptions)
  eval_df$Weight <- weight_map_F1[idx_F1] + weight_map_F2[idx_F2] +
    weight_map_F3[idx_F3]

  # E. Sort Records and Calculate Recall just before Precision makes a discrete
  # step below 95%
  eval_df <- eval_df[order(-eval_df$Weight), ]

  eval_df$cum_M <- cumsum(eval_df$True_Match == "Yes")
  eval_df$cum_U <- cumsum(eval_df$True_Match == "No")
  eval_df$precision <- eval_df$cum_M / (eval_df$cum_M + eval_df$cum_U)

  # Find max recall where precision is >= 0.95
  valid_rows <- eval_df[eval_df$precision >= 0.95, ]

  if(nrow(valid_rows) > 0) {
    # Divide by 1000 (Total M) to get the percentage recall
    current_grid$Recall[i] <- max(valid_rows$cum_M) / 1000
  } else {
    current_grid$Recall[i] <- 0
  }

  # Optional: Print progress every 20%
  # if(i %% 88 == 0) cat(sprintf("Progress: %d%%\n", round((i/441)*100)))
}
# Store the completed grid for this iteration
all_simulation_results[[sim]] <- current_grid
}

  # F. Aggregate the Monte Carlo Results
cat("\nAggregating Monte Carlo Results...\n")
```

```
##
## Aggregating Monte Carlo Results...
```

```r
final_results <- bind_rows(all_simulation_results) %>%
  group_by(rho_v, rho_m) %>%
  summarize(
    Mean_Recall = mean(Recall),
    Median_Recall = median(Recall), # Store in case comparison is desired
    .groups = 'drop'
  )
```

```r
# 4. Render the Contour Map
p <- ggplot(final_results, aes(x = rho_m, y = rho_v, z = Mean_Recall)) +
  geom_contour_filled(breaks = seq(0, 1, by = 0.1)) +
  geom_contour(color = "white", alpha = 0.5, breaks = seq(0, 1, by = 0.1)) +
  scale_x_continuous(expand = c(0, 0), breaks = seq(0, 0.8, by = 0.2)) +
  scale_y_continuous(expand = c(0, 0), breaks = seq(0, 0.8, by = 0.2)) +
  labs(
    title = "Linkage Algorithm Performance Contour",
    subtitle = "Mean Realized Recall at 95% Precision Boundary",
    x = expression("Correlation of Missingness (" * rho[m] * ")"),
    y = expression("Correlation of Recording Errors (" * rho[v] * ")"),
    fill = "Mean Recall"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    panel.grid = element_blank(),
    plot.title = element_text(face = "bold"),
    axis.title.x = element_text(margin = margin(t = 15)),
    axis.title.y = element_text(margin = margin(r = 15)),
    legend.position = "right",
    panel.border = element_rect(color = "black", fill = NA, size = 1)
  )

print(p)
```



From the contour map, it appears that the algorithm performs best when correlation in recording errors between fields is high and correlation in missingness between fields is low. While this result may seem surprising at first, it is consistent with the principles underlying this linkage algorithm. Recall that each field comparison outcome (e.g., "agree on Field 1" or "disagree on Field 2") contributes a certain "weight" to the calculation of a total log odds score for a given record pair. This total log odds score is simply the sum of the weights from each field. These weights were stored for each iteration of the above simulations in the vectors `weight_map_F1`, `weight_map_F2`, and `weight_map_F3`. To get a sense for the scale of these weights, we print the table below, which contains the values of these vectors from the most recent iteration of the last simulation (with $\rho_v = .80$, $\rho_m = .80$). Notice that agreement contributed a weight of roughly +2 to +3, while disagreement and missingness contributed weights close to -2 and zero, respectively. These weights stand up to a bit of intuitive examination. For example, the weight of zero for the event of missingness arises from the fact that missingness is equally likely among true matches as true non-matches, leading to an odds score of 1 and a log odds of zero.

```
weight_mat <- matrix(c(weight_map_F1, weight_map_F2, weight_map_F3), ncol = 3)
colnames(weight_mat) <- c("F1", "F2", "F3")
rownames(weight_mat) <- c("Agree", "Disagree", "Missing")
knitr::kable(weight_mat, digits = 2, caption = "Empirical Log-Odds Weights")
```

Table 1: Empirical Log-Odds Weights

	F1	F2	F3
Agree	2.90	2.89	2.31
Disagree	-2.00	-2.06	-1.96
Missing	0.04	0.01	0.05

The optimal region in the contour plot can likely be explained by the fact that while correlation in the events of recording errors and missingness was allowed to change, the marginal probabilities of those events remained fixed ($\lambda_v = 0.2$ and $\lambda_m = 0.1$). Thus, when correlation in recording errors was high, there were a handful of true matches that suffered from multiple such errors, while the majority of true matches were error-free. Thus, there were a few "sacrificial" records that received low log odds scores but allowed most of the true matches to rise to the top of the distribution of scores. Why did the same pattern not hold for the event of missingness? Because missingness contributes a neutral weight of zero, rather than a heavy penalty of -2, a true match can easily survive a single missing field and still clear the precision threshold (which was a score of 3.83 on the last iteration of the final simulation) on the strength of its remaining fields. Spreading missingness allows the maximum number of true matches to survive, whereas concentrating it creates catastrophic scores of zero that plunge those records below the threshold.

Limitations

Notable assumptions on which this simulation relies, besides that of reasonable data quality (marginal probabilities of recording errors and missingness equal to 0.2 and 0.1, respectively), include: 1) a favorable post-blocking ratio can be achieved between true non-matches and true matches (e.g., 19:1), 2) the missingness in the target file is not dependent upon its recording errors, and 3) the distributions of the fields used to link records are not too unlike the synthetic ones used in this simulation, which had a few common values and several uncommon ones.

Conclusion

When marginal probabilities of recording errors and missingness in a target data file equal 0.2 and 0.1, respectively, the results of our simulation suggest that the linkage algorithm performs well in spite of A)

high correlation in recording errors between fields, and B) high correlation in missingness between fields. This is notable in light of the fact that (A) and (B) violate the core independence assumptions used to build the algorithm. In fact, moderate to high correlation leads to better performance when the correlation exists in both recording errors and missingness, or in recording errors alone.

The optimal scenario for the linkage algorithm occurs when high correlation in recording errors coexists with low correlation in missingness. In this case, most true matches observe multiple fields correctly and rise to the top of the distribution of log odds scores at the expense of a few error-heavy, "sacrificial" records. Simultaneously, because missingness is mathematically neutral rather than penalizing, few true matches are lost to independent missingness across fields. The main takeaway of this paper is that, assuming overall data quality is reasonable, organizations operating in the "anchor" scenario need not let high correlation in recording errors and missingness between fields in a target file deter them from using the probabilistic linkage algorithm described by David White.